

Telemetry system for E-Racing Bergamo

Simone Manenti 1048788
Cattaneo Federico 1053265

May 9, 2024

Introduzione

Il progetto

Il progetto prevede la realizzazione di un'applicazione Java che permetta di ricevere, visualizzare e archiviare i dati di telemetria provenienti dalla vettura ERB-01, una monoposto da competizione sportiva progettata dal team E-Racing Bergamo.

Descrizione generale

Prospettive del prodotto:

Il prodotto è progettato per essere utilizzato su un comune PC dotato di Windows e Java 21. Non ci sono particolari esigenze per quanto riguarda la potenza di calcolo richiesta per la sua esecuzione.

L'archivio nel database interno richiede almeno 100MB di spazio su disco.

Il sistema prevede l'interfacciamento con una periferica esterna, chiamata semplicemente ricevitore, tramite porta USB.

Il ricevitore è un dispositivo sviluppato separatamente dal team E-Racing Bergamo, le cui specifiche sono note.

Funzionalità del prodotto

- Connettersi e ricevere correttamente i dati provenienti dal ricevitore.
- Archiviare i dati ricevuti all'interno del database locale.
- Visualizzare i dati in modo intuitivo tramite interfaccia grafica.
- Esportare i dati su file .csv per archiviazione o per ulteriori analisi

Caratteristiche dell'utente:

L'applicazione è rivolta esclusivamente al personale del team E-Racing Bergamo, in particolare il sistema è sviluppato appositamente come strumento di supporto per il dipartimento di "Controllo" che si occupa della programmazione e dello sviluppo dei sistemi elettronici della monoposto.

Si prevede quindi che il personale sia pienamente a conoscenza del dominio al quale l'applicazione appartiene.

Vincoli

Tutte le funzionalità offerte dall'applicazione sono a disposizione dell'utente senza la necessità di autenticarsi.

Benché sia possibile collegare fisicamente più unità riceventi allo stesso PC, l'applicazione consente un unico canale di comunicazione per volta.

Specifica dei requisiti

Scopo

Il documento esplicita i requisiti del sistema di raccolta dati

3.0.1 Obiettivo

L'obiettivo del prodotto è quello di velocizzare le operazioni di download dei dati telemetrici, evitando il collegamento via cavo ad auto ferma, e di permettere l'accesso ai dati quando il mezzo è in movimento.

3.0.2 Definizioni, acronimi, abbreviazioni

- **TD:** Telemetry Data indica il frame contenente le informazioni sullo stato del mezzo.
- **TX:** Indica l'unità hardware a bordo del mezzo responsabile dell'invio dei dati via wireless verso l'unità ricevente.
- **RX:** Indica l'unità hardware, collegata al PC su cui è in esecuzione il software di telemetria, responsabile della ricezione dei dati.
- **COM:** Porta seriale attraverso il quale avviene lo scambio dati tra l'applicazione e l'unità ricevente.

Requisiti di interfacce esterne:

Lo scopo dell'applicazione è quello di fornire una panoramica generale sullo stato della monoposto di conseguenza l'interfaccia grafica deve riflettere la struttura di un "pannello di controllo" virtuale: la totalità delle informazioni e delle funzionalità devono essere accessibili in un'unica schermata in modo compatto e facilmente interpretabile.

Requisiti funzionali

- Connessione al ricevitore: La selezione del ricevitore avviene tramite menu a discesa, l'apertura e la chiusura della comunicazione si ottiene attraverso un pulsante.
- Archiviazione dati: Il salvataggio dei dati deve essere eseguito in modo automatico durante tutto l'arco temporale in cui la comunicazione è aperta.

Requisiti di prestazioni

- Il sistema supporta un singolo accesso simultaneo ad un'unità ricevente.
- Il database deve archiviare fino a 100 000 registrazioni di TD.
- Il sistema deve visualizzare i dati telemetrici real-time entro 1000ms dalla loro trasmissione da parte dell'unità trasmittente.
- Il sistema deve essere in grado di gestire una frequenza di almeno 2 TD al secondo.

Vincoli di design

L'interfacciamento con l'unità ricevente è vincolante per il funzionamento dell'applicazione.

Requisiti di robustezza e affidabilità

Un malfunzionamento del software non costituisce fattore di rischio per cose o persone per cui non ci sono requisiti minimi di robustezza e affidabilità.

Ingegneria dei Requisiti

Il processo di sviluppo agile ci ha consentito di non focalizzarci eccessivamente nel tentativo di stabilire i requisiti nella fase iniziale del progetto, consapevoli del fatto che questi si sarebbero palesati in corso d'opera.

I requisiti, infatti, si sono evoluti insieme al progetto.

Le tecniche utilizzate per la determinazione dei requisiti sono state:

- **Brainstorming:** Utilizzato principalmente nella fase iniziale, ci ha fornito un insieme di idee che ci ha permesso di inquadrare una prima direzione di sviluppo.
- **Analisi di scenari:** Attraverso scenari ipotetici di utilizzo abbiamo cercato di identificare le caratteristiche chiave per il nostro prodotto.
- **Prototipazione:** La prototipazione ci ha consentito, fin dalle fasi iniziali, di comprendere se i requisiti iniziali fossero nella portata delle nostre capacità e il se progetto fosse effettivamente realizzabile.
- **Dialogo:** Il dialogo con i membri del team è stato fondamentale durante tutto l'arco di sviluppo del software.

I requisiti sono stati classificati attraverso la tecnica MoSCoW.

4.0.1 Must Have

- Gestione porta COM comunicazione con ricevitore.
- Gestione database locale per archiviazione/consultazione storico dati.
- GUI: Visualizzazione real-time dei dati ricevuti.
- GUI: Gestione porta COM.
- GUI: Visualizzazione tabellare storico dati.
- GUI: Visualizzazione errori di connessione.
- Export dello storico dati tramite file CSV.
- Ricevere dati in tempo reale dalla vettura.

4.0.2 Should have

- GUI: identificazione tramite colore dei valori che richiedono l'attenzione dell'operatore.
- Accesso utenti.
- Visualizzazione dati tramite grafico real-time.

4.0.3 Could have

- Possibilità di muovere i componenti grafici all'interno dell'interfaccia.
- Possibilità di cambiare il tema dell'interfaccia.

4.0.4 Would have

- Possibilità di inviare comandi alla monoposto.

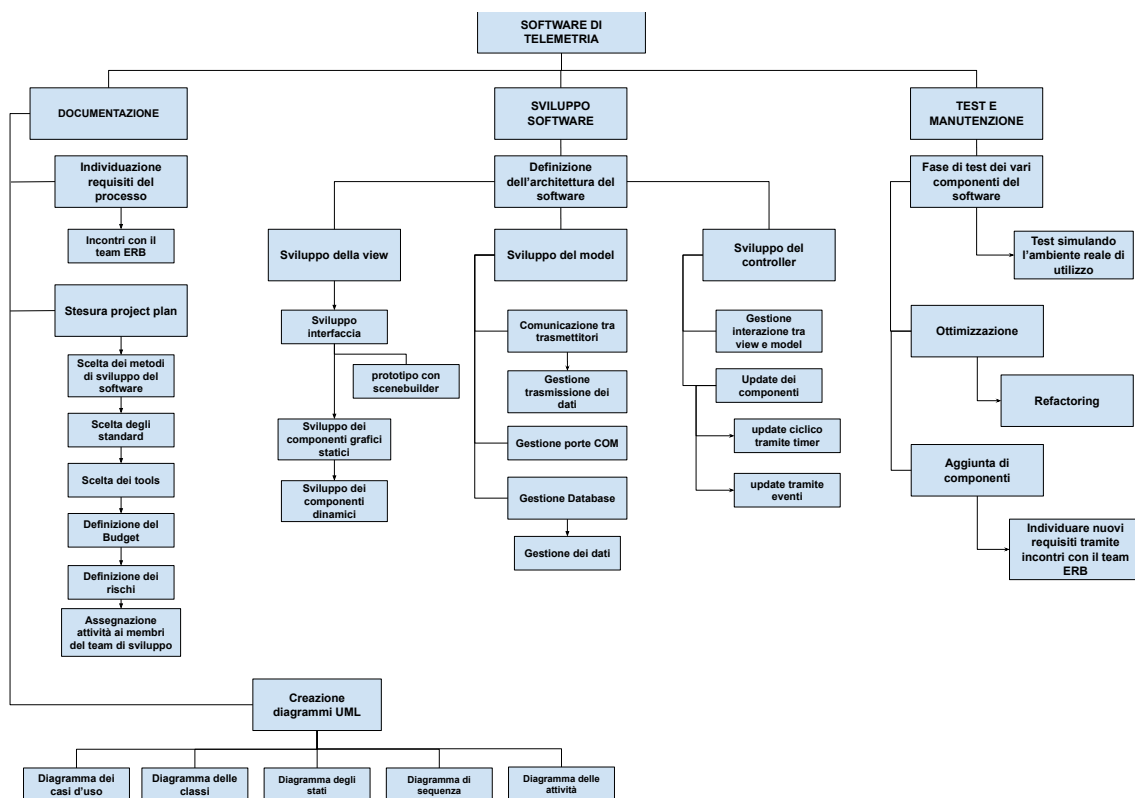
Gestione del personale

Il team di sviluppo è composto da 2 persone, di pari competenze, senza gerarchie interne.

I membri del team sono anche parte della squadra che sarà l'utente del sistema.

Di conseguenza il team interpreta due ruoli distinti all'interno dell'organizzazione Scrum.

Prima dello sprint, il team si occupa di scrivere e apportare modifiche al backlog adottando il ruolo di Product Owner, per la stesura del backlog il team prima ha creato il diagramma "work breakdown structure (WBS)", nel quale il progetto è stato diviso in diverse attività da svolgere, seguendo l'approccio top-down.



Dopodiché i membri agiscono separatamente o congiuntamente nella realizzazione degli obiettivi selezionati nello sprint planning e indipendentemente dalla modalità di lavoro, i membri sono sempre a stretto contatto e il confronto è continuo.

Ciclo di vita del software

La natura del progetto e la mancanza di esperienza del team nell'ambito dei software telemetrici hanno suggerito l'adozione di un approccio agile.

In generale il metodo utilizzato è SCRUM al quale sono state affiancate tecniche provenienti dall'eXtreme Programming, nello specifico la programmazione in coppia e la proprietà collettiva.

Nelle prime fasi dello sviluppo la prototipazione ha svolto un ruolo fondamentale per due motivi: in primo luogo ha permesso di verificare la fattibilità di alcune soluzioni tecniche alla base del progetto e in secondo luogo ha gettato le fondamenta sulle quali si è evoluto il software.

La scelta di un modello iterativo a cicli brevi ha consentito di adattare i requisiti alle reali esigenze del team durante lo sviluppo e il collaudo del software.

Particolare importanza hanno avuto i test, eseguiti subito dopo l'implementazione di una nuova funzionalità, non solo come strumento di verifica del corretto funzionamento ma anche come strumento di confronto tra le aspettative e il reale comportamento dell'applicazione.

Gestione della configurazione

La gestione della configurazione è stata affidata interamente alla piattaforma GitHub.

La linea guida che il team di sviluppo ha seguito è stata quella di mantenere il main branch sempre funzionante e testato.

Ad esclusione delle modifiche minori per cui è stato eseguito un semplice commit sul main branch per tutte le altre modifiche il processo di sviluppo ha previsto la creazione di un branch secondario, l'implementazione delle modifiche o delle correzioni, la verifica attraverso i test e, in conclusione, il merge con il Main.

Per l'approvazione del Merge o per la risoluzione di issue, venivano coinvolti entrambi i membri per poter avere un confronto diretto e velocizzare il processo

In molti casi, per scopi didattici o per semplice curiosità, il team ha creato dei “Dead branch” in cui il team ha sperimentato funzionalità specifiche al fine di valutarne la complessità e l'integrabilità nel progetto.

Organizzazione della repository: Abbiamo organizzato la repository nel seguente modo:

- **Arduino:** Contiene i firmware dei microcontrollori che usiamo come trasmettitore e ricevitore
- **Code/telemetry:** contiene il codice del software
- **Documentazione:** Contiene tutta la documentazione relativa al progetto: Project Plan, Sprint e documentazione (nella quale descriviamo in maniera più dettagliata il nostro progetto)
- **Img:** contiene le immagini inserite nella pagina di github

Qualità del software

Per la qualità edl software ci siamo basati sullo standard ISO/IEC9126, suddiviso in requisiti funzionali e requisiti di qualità.

8.0.1 Requisiti funzionali

Il requisito presente in questa sezione è il requisito di **Funzionalità**, secondo il quale il nostro software deve operare seguendo delle condizioni di:

- **Appropriatezza:** cioè fornire funzioni che soddisfino i compiti e gli obiettivi prefissati all'utente. Ad esempio uno degli obiettivi era quello di fornire una funzione per la visualizzazione in tempo reale dei dati, e questo è stato soddisfatto.
- **Accuratezza:** cioè fornire i risultati concordati. Rimanendo sulla visualizzazione dei dati in tempo reale, era la capacità di visualizzare la velocità dell'auto in tempo reale in maniera corretta.
- **Interoperabilità:** capacità del sistema di operare con uno o più sistemi specificati. Sempre con l'esempio precedente lo possiamo vedere come la capacità del software di interagire con il ricevitore per potere gestire i dati e rappresentarli a schermo.
- **Conformità:** capacità del software di aderire a standard, convenzioni e regolamenti rilevanti al settore operativo a cui vengono applicate. Questo lo possiamo vedere come la capacità del software di rappresentare i dati entro il limite imposto di 200ms.
- **Sicurezza:** Capacità del prodotto software di proteggere informazioni e dati. Questo è implicitamente presente poichè i dati vengono salvati su un database locale e solo chi ha la password del computer può accedere ai dati.

8.0.2 Requisiti di qualità

In questa sezione sono presenti diversi requisiti:

- **Affidabilità:** Capacità di mantenere un determinato livello di prestazioni quando usato in determinate condizioni per un determinato periodo. Questo è garantito grazie ai test effettuati durante la programmazione del software.
- **Efficienza:** Capacità di fornire appropriate prestazioni relativamente alla quantità di risorse usate.
- **Usabilità:** Capacità del software di essere capito, appreso, usato e accettato dall'utente, tramite condizioni di:
 - Comprensibilità:** l'utente è in grado di comprendere se il software è appropriato, questo è accertato dal fatto che lo sviluppo è stato fatto a stretto contatto con l'utente finale e ogni versione e funzionalità aggiunta al software sono state concordate con l'utente.
 - Apprendibilità:** capacità di ridurre l'impegno richiesto agli utenti per imparare ad usare l'applicazione, questo è garantito dal fatto che il software è stato sviluppato con un'interfaccia semplice e con solo funzionalità essenziali, che lo rendono "plug and play".
 - Operabilità:** capacità di mettere in condizione gli utenti di farne uso per i propri scopi, questo è garantito sempre dal fatto che lo sviluppo è avvenuto a stretto contatto con gli utenti.
 - Attrattiva:** capacità del software di essere piacevole per l'utente, questo è garantito dall'interfaccia semplice e funzionale allo scopo.
- **Manutenibilità:** capacità del software di essere modificato, includendo correzioni, miglioramenti o adattamenti.
 - Analizzabilità:** Il software deve essere facile da analizzare per localizzare un errore, questo è possibile grazie all'architettura utilizzata per lo sviluppo del software.
 - Modificabilità:** capacità di implementare una modifica e di sostituire componenti, questo è possibile grazie alla divisione in classi "indipendenti" e all'uso delle Pane. Ad esempio se dovessimo sostituire il contenuto di una Tab non dovremmo modificare la classe view o altre classi.
 - Stabilità e testabilità:** evitare effetti inaspettati derivanti da modifiche errate e capacità di testare queste modifiche.
- **Portabilità:** capacità del software di essere trasportato da un ambiente di lavoro ad un altro. Il software deve essere **Adattabile, installabile, conforme e sostituibile:** quindi il nostro software deve poter essere adattato per diversi ambienti operativi, di aderire a standard e convenzioni relative alla portabilità e di essere utilizzato al posto di un altro software per svolgere gli stessi compiti nello stesso ambiente. Questo è garantito se l'utente andrà ad utilizzare sempre dei microcontrollori programmabili e installerà il firmware definito nella pagina di github e se utilizzerà sensori definiti nel software.

Design e Architettura del software

9.0.1 Design

Tra i Design Pattern utilizzati nel progetto si riconoscono:

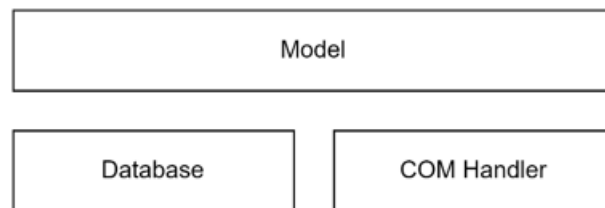
- **singleton pattern:** Per garantire l'unicità della classe che gestisce il database.
- **Builder pattern:** Abbiamo avuto modo di interagire con questo pattern nella creazione di istanze di oggetti "Gauge".
- **Observable-Observer like pattern:** Utilizzato per evitare un richiamo diretto di una funzione del View su un evento di ricezione dati all'interno del Model.

Abbiamo utilizzato i seguenti design pattern perchè ci hanno permesso di migliorare la comprensione del codice e di permetterci di avere una riusabilità dei componenti e una migliore manutenibilità.

9.0.2 Stile architetturale

Lo stile architetturale adottato è MVC – Model View Controller per beneficiare del vantaggio di disaccoppiare l'interfaccia grafica dalla logica di programma.

- **View:** gestisce tutta l'interfaccia grafica basata su JavaFX.
- **Model:** gestisce tutte le funzionalità logiche del programma. All'interno del Model è riconoscibile una semplice struttura a strati per cui il model si appoggia sui servizi sottostanti forniti dal gestore del database e dal gestore della porta seriale.
- **Controller:** funge da intermediario tra Model e View: si occupa di richiamare le funzioni del Model corrispondenti ai comandi del View e gestisce l'aggiornamento della GUI con i dati provenienti dal Model.

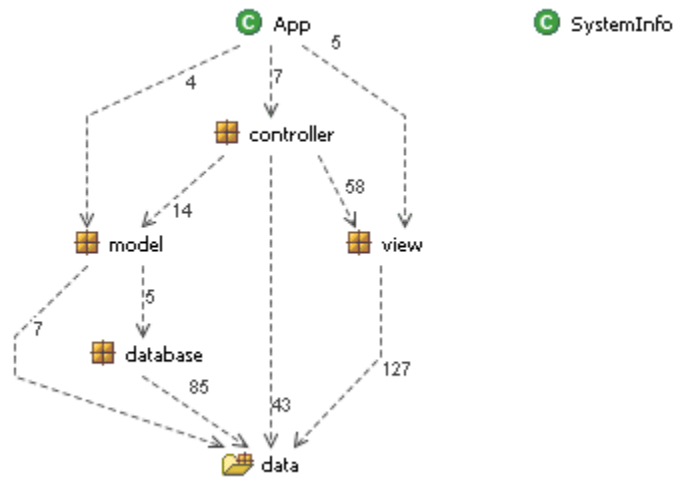


9.0.3 Analisi delle dipendenze

L'analisi della struttura del progetto e delle dipendenze tra le classi è stata effettuata attraverso il tool Structure101.

Questo tool si è rivelato efficace nel fornire una panoramica delle relazioni tra i moduli che compongono il programma.

Inoltre, ha evidenziato alcune criticità nelle dipendenze tra le classi che difficilmente avremmo rilevato senza il suo utilizzo.



Complementare a Structure101 è stato il processo di refactoring con i tool offerti da Eclipse grazie ai quali sono state apportate modifiche all'organizzazione delle classi con un impatto minimo sulle tempistiche di programmazione.

Manutenzione e Test

Durante la vita del software abbiamo incontrato e incontreremo diverse forme di manutenzione.

Correttiva: Rimozione di bug. Ad esempio dopo aver fatto i test ci siamo accorti che il Parsing divideva la stringa usando un simbolo errato rispetto a quello che volevamo utilizzare.

Adattiva: Manutenzione fatta per l'adattamento a cambiamenti ambientali. Dopo aver sostituito javafx17 con la versione di javafx21, il grafico non plottava più i dati inseriti. Questo era dovuto all'incompatibilità della versione di javafx con le funzioni utilizzate. Abbiamo così provveduto a modificare le funzioni per permettere il plot dei dati sul grafico.

Perfettiva: Manutenzione fatta ad ogni sprint, dopo aver dialogato con l'utente, nel caso in cui fossero apparsi nuovi requisiti o richieste di modifiche del software.

Preventiva: Manutenzione fatta per garantire una manutenzione ottimale dopo aver distribuito il software all'utente.

Il nostro software, essendo sviluppato da alcuni componenti del team E-Racing Bergamo, non può essere considerato un componente COTS, poichè gli utenti finali sono parte del team che ha commissionato il lavoro, e anche la manutenzione viene svolta da una parte degli stessi utenti che utilizzeranno il software.

Essendo gli utenti finali parte del team che ha sviluppato il software, ed avendo un'interfaccia semplici con funzionalità essenziali, non consegniamo una documentazione relativa alle "istruzioni di utilizzo" ma consegniamo solo la documentazione relativa alla pianificazione e progettazione del software, per semplificare future attività di manutenzione relative ad aggiornamenti di funzionalità o relative a correzioni del codice.

Test

Per svolgere i test abbiamo utilizzato il framework junit4, testando la parte di logica del software.

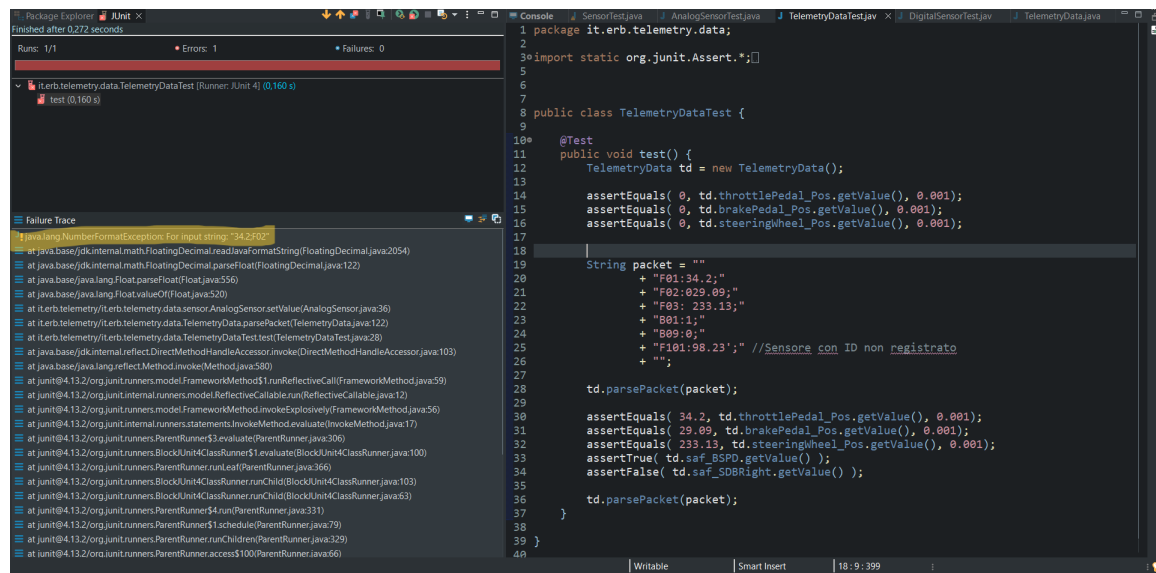
Il test della parte grafica, nella quale sono presenti componenti di javaFx, è possibile testarla implementando l'API TestFx in combinazione con Junit4.

Purtroppo l'adozione di javafx21 ci ha impedito di fare test automatici sull'interfaccia grafica.

Abbiamo così deciso di implementare, per la selezione delle date, dei popup di errore per evitare che l'utente scelga date incompatibili, e parallelamente, per darci un feedback nel capire come si comporti il software in ogni casistica di selezione delle date e verificare che il software si comporti sempre come aspettato.

10.0.1 TelemetryDataTest

In questa classe di test abbiamo riscontrato un errore durante l'esecuzione del junitTest:

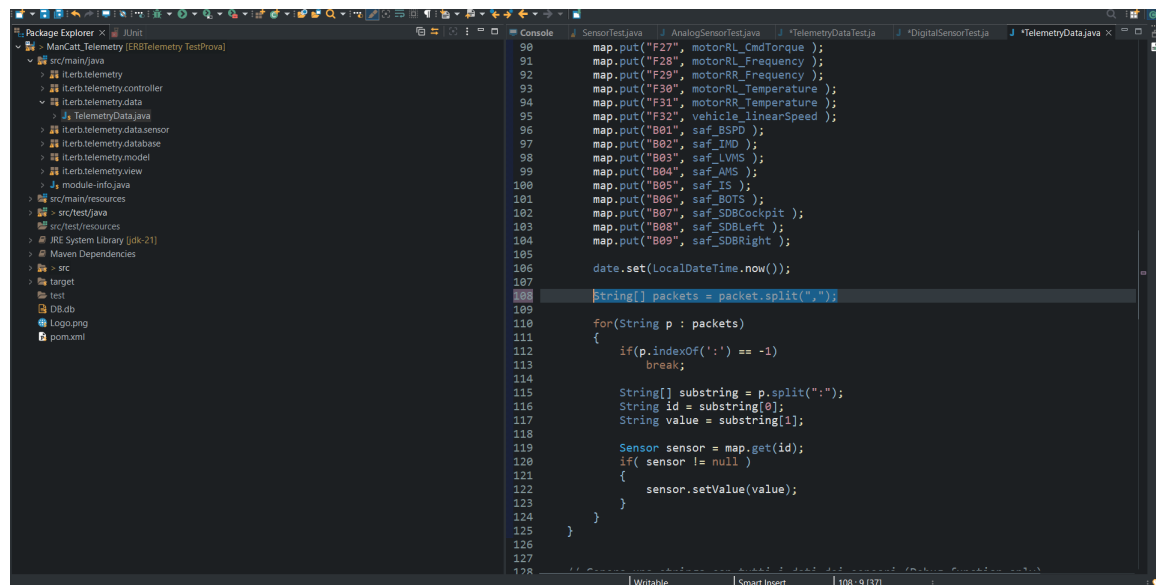


```
1 package it.erb.telemetry.data;
2
3 import static org.junit.Assert.*;
4
5
6
7
8 public class TelemetryDataTest {
9
10     @Test
11     public void test() {
12         TelemetryData td = new TelemetryData();
13
14         assertEquals( 0, td.throttlePedal_Pos.getValue(), 0.001);
15         assertEquals( 0, td.brakePedal_Pos.getValue(), 0.001);
16         assertEquals( 0, td.steeringWheel_Pos.getValue(), 0.001);
17
18
19         String packet = ""
20             + "F01:34.2;"
21             + "F02:029.09;"
22             + "F03:233.13;"
23             + "B01:1;"
24             + "B09:0;"
25             + "F101:98.23;" //Sensore con ID non registrato
26             + ";";
27
28         td.parsePacket(packet);
29
30         assertEquals( 34.2, td.throttlePedal_Pos.getValue(), 0.001);
31         assertEquals( 29.09, td.brakePedal_Pos.getValue(), 0.001);
32         assertEquals( 233.13, td.steeringWheel_Pos.getValue(), 0.001);
33         assertTrue( td.saf_BSPD.getValue() );
34         assertEquals( td.saf_SDBRight.getValue() );
35
36         td.parsePacket(packet);
37     }
38 }
39
40
```

Failure Trace

```
java.lang.NumberFormatException: For input string: "34.2;02"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.base/java.lang.Float.parseFloat(Float.java:556)
    at java.base/java.lang.Float.parseFloat(Float.java:520)
    at it.erb.telemetry.data.sensor.AnalogSensor.setValue(AnalogSensor.java:36)
    at it.erb.telemetry.data.TelemetryData.parsePacket(TelemetryData.java:122)
    at it.erb.telemetry.data.TelemetryDataTest.test(TelemetryDataTest.java:28)
    at java.base/java.lang.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
    at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
    at org.junit.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
    at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
    at org.junit.runners.model.Statement.invokeMethod.evaluate(InvokeMethod.java:17)
    at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
    at org.junit.runners.BlockJUnit4ClassRunner$1.evaluate(BlockJUnit4ClassRunner.java:100)
    at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:103)
    at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:63)
    at org.junit.runners.ParentRunner$4.run(ParentRunner.java:331)
    at org.junit.runners.ParentRunner$1.schedule(ParentRunner.java:79)
    at org.junit.runners.ParentRunner.runChildren(ParentRunner.java:329)
    at org.junit.runners.ParentRunner.access$100(ParentRunner.java:66)
```

Il software si sarebbe aspettato l'utilizzo del simbolo ";" per la divisione dei dati all'interno dei pacchetti, ma nel codice, nella classe TelemetryData, era stato utilizzato il simbolo "," per la divisione dei pacchetti.



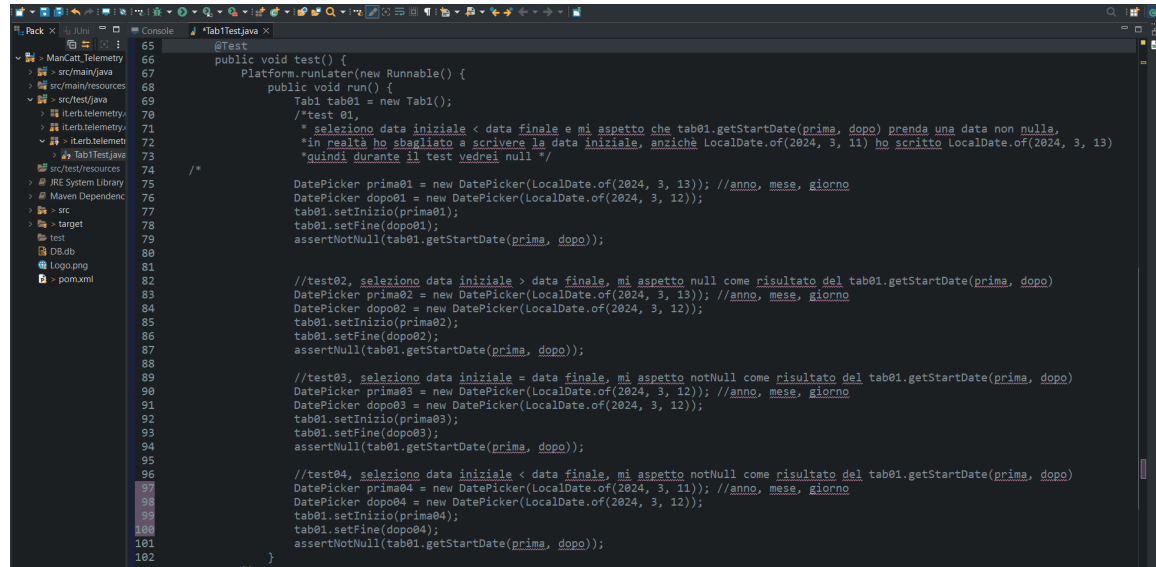
```
1 package it.erb.telemetry;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class TelemetryData {
7
8     private Map<String, Sensor> map;
9
10     public TelemetryData() {
11         map = new HashMap<>();
12     }
13
14     public void parsePacket(String packet) {
15         String[] packets = packet.split(",");
16
17         for (String p : packets) {
18             if (p.indexOf(':') == -1)
19                 break;
20
21             String[] substring = p.split(":");
22             String id = substring[0];
23             String value = substring[1];
24
25             Sensor sensor = map.get(id);
26             if (sensor != null) {
27                 sensor.setValue(value);
28             }
29         }
30     }
31
32     public void setMap(Map<String, Sensor> map) {
33         this.map = map;
34     }
35
36     public Map<String, Sensor> getMap() {
37         return map;
38     }
39
40     public void setDate(LocalDate date) {
41         this.date = date;
42     }
43
44     public LocalDate getDate() {
45         return date;
46     }
47
48     public void setVehicleLinearSpeed(float speed) {
49         this.vehicleLinearSpeed = speed;
50     }
51
52     public float getVehicleLinearSpeed() {
53         return vehicleLinearSpeed;
54     }
55
56     public void setSaf_BSPD(boolean saf_BSPD) {
57         this.saf_BSPD = saf_BSPD;
58     }
59
60     public boolean getSaf_BSPD() {
61         return saf_BSPD;
62     }
63
64     public void setSaf_IMD(boolean saf_IMD) {
65         this.saf_IMD = saf_IMD;
66     }
67
68     public boolean getSaf_IMD() {
69         return saf_IMD;
70     }
69
70     public void setSaf_LNMS(boolean saf_LNMS) {
71         this.saf_LNMS = saf_LNMS;
72     }
73
74     public boolean getSaf_LNMS() {
75         return saf_LNMS;
76     }
77
78     public void setSaf_AMS(boolean saf_AMS) {
79         this.saf_AMS = saf_AMS;
80     }
81
82     public boolean getSaf_AMS() {
83         return saf_AMS;
84     }
85
86     public void setSaf_IS(boolean saf_IS) {
87         this.saf_IS = saf_IS;
88     }
89
90     public boolean getSaf_IS() {
91         return saf_IS;
92     }
93
94     public void setSaf_BOTS(boolean saf_BOTS) {
95         this.saf_BOTS = saf_BOTS;
96     }
97
98     public boolean getSaf_BOTS() {
99         return saf_BOTS;
100     }
101
102     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
103         this.saf_SDB Cockpit = saf_SDB Cockpit;
104     }
105
106     public boolean getSaf_SDB Cockpit() {
107         return saf_SDB Cockpit;
108     }
109
110     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
111         this.saf_SDBLeft = saf_SDBLeft;
112     }
113
114     public boolean getSaf_SDBLeft() {
115         return saf_SDBLeft;
116     }
117
118     public void setSaf_SDBRight(boolean saf_SDBRight) {
119         this.saf_SDBRight = saf_SDBRight;
120     }
121
122     public boolean getSaf_SDBRight() {
123         return saf_SDBRight;
124     }
125
126     public void setDate(LocalDate date) {
127         this.date = date;
128     }
129
130     public LocalDate getDate() {
131         return date;
132     }
133
134     public void setVehicleLinearSpeed(float speed) {
135         this.vehicleLinearSpeed = speed;
136     }
137
138     public float getVehicleLinearSpeed() {
139         return vehicleLinearSpeed;
140     }
141
142     public void setSaf_BSPD(boolean saf_BSPD) {
143         this.saf_BSPD = saf_BSPD;
144     }
145
146     public boolean getSaf_BSPD() {
147         return saf_BSPD;
148     }
149
150     public void setSaf_IMD(boolean saf_IMD) {
151         this.saf_IMD = saf_IMD;
152     }
153
154     public boolean getSaf_IMD() {
155         return saf_IMD;
156     }
157
158     public void setSaf_LNMS(boolean saf_LNMS) {
159         this.saf_LNMS = saf_LNMS;
160     }
161
162     public boolean getSaf_LNMS() {
163         return saf_LNMS;
164     }
165
166     public void setSaf_AMS(boolean saf_AMS) {
167         this.saf_AMS = saf_AMS;
168     }
169
170     public boolean getSaf_AMS() {
171         return saf_AMS;
172     }
173
174     public void setSaf_IS(boolean saf_IS) {
175         this.saf_IS = saf_IS;
176     }
177
178     public boolean getSaf_IS() {
179         return saf_IS;
180     }
181
182     public void setSaf_BOTS(boolean saf_BOTS) {
183         this.saf_BOTS = saf_BOTS;
184     }
185
186     public boolean getSaf_BOTS() {
187         return saf_BOTS;
188     }
189
190     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
191         this.saf_SDB Cockpit = saf_SDB Cockpit;
192     }
193
194     public boolean getSaf_SDB Cockpit() {
195         return saf_SDB Cockpit;
196     }
197
198     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
199         this.saf_SDBLeft = saf_SDBLeft;
200     }
201
202     public boolean getSaf_SDBLeft() {
203         return saf_SDBLeft;
204     }
205
206     public void setSaf_SDBRight(boolean saf_SDBRight) {
207         this.saf_SDBRight = saf_SDBRight;
208     }
209
210     public boolean getSaf_SDBRight() {
211         return saf_SDBRight;
212     }
213
214     public void setDate(LocalDate date) {
215         this.date = date;
216     }
217
218     public LocalDate getDate() {
219         return date;
220     }
221
222     public void setVehicleLinearSpeed(float speed) {
223         this.vehicleLinearSpeed = speed;
224     }
225
226     public float getVehicleLinearSpeed() {
227         return vehicleLinearSpeed;
228     }
229
230     public void setSaf_BSPD(boolean saf_BSPD) {
231         this.saf_BSPD = saf_BSPD;
232     }
233
234     public boolean getSaf_BSPD() {
235         return saf_BSPD;
236     }
237
238     public void setSaf_IMD(boolean saf_IMD) {
239         this.saf_IMD = saf_IMD;
240     }
241
242     public boolean getSaf_IMD() {
243         return saf_IMD;
244     }
245
246     public void setSaf_LNMS(boolean saf_LNMS) {
247         this.saf_LNMS = saf_LNMS;
248     }
249
250     public boolean getSaf_LNMS() {
251         return saf_LNMS;
252     }
253
254     public void setSaf_AMS(boolean saf_AMS) {
255         this.saf_AMS = saf_AMS;
256     }
257
258     public boolean getSaf_AMS() {
259         return saf_AMS;
260     }
261
262     public void setSaf_IS(boolean saf_IS) {
263         this.saf_IS = saf_IS;
264     }
265
266     public boolean getSaf_IS() {
267         return saf_IS;
268     }
269
270     public void setSaf_BOTS(boolean saf_BOTS) {
271         this.saf_BOTS = saf_BOTS;
272     }
273
274     public boolean getSaf_BOTS() {
275         return saf_BOTS;
276     }
277
278     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
279         this.saf_SDB Cockpit = saf_SDB Cockpit;
280     }
281
282     public boolean getSaf_SDB Cockpit() {
283         return saf_SDB Cockpit;
284     }
285
286     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
287         this.saf_SDBLeft = saf_SDBLeft;
288     }
289
290     public boolean getSaf_SDBLeft() {
291         return saf_SDBLeft;
292     }
293
294     public void setSaf_SDBRight(boolean saf_SDBRight) {
295         this.saf_SDBRight = saf_SDBRight;
296     }
297
298     public boolean getSaf_SDBRight() {
299         return saf_SDBRight;
300     }
301
302     public void setDate(LocalDate date) {
303         this.date = date;
304     }
305
306     public LocalDate getDate() {
307         return date;
308     }
309
310     public void setVehicleLinearSpeed(float speed) {
311         this.vehicleLinearSpeed = speed;
312     }
313
314     public float getVehicleLinearSpeed() {
315         return vehicleLinearSpeed;
316     }
317
318     public void setSaf_BSPD(boolean saf_BSPD) {
319         this.saf_BSPD = saf_BSPD;
320     }
321
322     public boolean getSaf_BSPD() {
323         return saf_BSPD;
324     }
325
326     public void setSaf_IMD(boolean saf_IMD) {
327         this.saf_IMD = saf_IMD;
328     }
329
330     public boolean getSaf_IMD() {
331         return saf_IMD;
332     }
333
334     public void setSaf_LNMS(boolean saf_LNMS) {
335         this.saf_LNMS = saf_LNMS;
336     }
337
338     public boolean getSaf_LNMS() {
339         return saf_LNMS;
340     }
341
342     public void setSaf_AMS(boolean saf_AMS) {
343         this.saf_AMS = saf_AMS;
344     }
345
346     public boolean getSaf_AMS() {
347         return saf_AMS;
348     }
349
350     public void setSaf_IS(boolean saf_IS) {
351         this.saf_IS = saf_IS;
352     }
353
354     public boolean getSaf_IS() {
355         return saf_IS;
356     }
357
358     public void setSaf_BOTS(boolean saf_BOTS) {
359         this.saf_BOTS = saf_BOTS;
360     }
361
362     public boolean getSaf_BOTS() {
363         return saf_BOTS;
364     }
365
366     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
367         this.saf_SDB Cockpit = saf_SDB Cockpit;
368     }
369
370     public boolean getSaf_SDB Cockpit() {
371         return saf_SDB Cockpit;
372     }
373
374     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
375         this.saf_SDBLeft = saf_SDBLeft;
376     }
377
378     public boolean getSaf_SDBLeft() {
379         return saf_SDBLeft;
380     }
381
382     public void setSaf_SDBRight(boolean saf_SDBRight) {
383         this.saf_SDBRight = saf_SDBRight;
384     }
385
386     public boolean getSaf_SDBRight() {
387         return saf_SDBRight;
388     }
389
390     public void setDate(LocalDate date) {
391         this.date = date;
392     }
393
394     public LocalDate getDate() {
395         return date;
396     }
397
398     public void setVehicleLinearSpeed(float speed) {
399         this.vehicleLinearSpeed = speed;
400     }
401
402     public float getVehicleLinearSpeed() {
403         return vehicleLinearSpeed;
404     }
405
406     public void setSaf_BSPD(boolean saf_BSPD) {
407         this.saf_BSPD = saf_BSPD;
408     }
409
410     public boolean getSaf_BSPD() {
411         return saf_BSPD;
412     }
413
414     public void setSaf_IMD(boolean saf_IMD) {
415         this.saf_IMD = saf_IMD;
416     }
417
418     public boolean getSaf_IMD() {
419         return saf_IMD;
420     }
421
422     public void setSaf_LNMS(boolean saf_LNMS) {
423         this.saf_LNMS = saf_LNMS;
424     }
425
426     public boolean getSaf_LNMS() {
427         return saf_LNMS;
428     }
429
430     public void setSaf_AMS(boolean saf_AMS) {
431         this.saf_AMS = saf_AMS;
432     }
433
434     public boolean getSaf_AMS() {
435         return saf_AMS;
436     }
437
438     public void setSaf_IS(boolean saf_IS) {
439         this.saf_IS = saf_IS;
440     }
441
442     public boolean getSaf_IS() {
443         return saf_IS;
444     }
445
446     public void setSaf_BOTS(boolean saf_BOTS) {
447         this.saf_BOTS = saf_BOTS;
448     }
449
450     public boolean getSaf_BOTS() {
451         return saf_BOTS;
452     }
453
454     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
455         this.saf_SDB Cockpit = saf_SDB Cockpit;
456     }
457
458     public boolean getSaf_SDB Cockpit() {
459         return saf_SDB Cockpit;
460     }
461
462     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
463         this.saf_SDBLeft = saf_SDBLeft;
464     }
465
466     public boolean getSaf_SDBLeft() {
467         return saf_SDBLeft;
468     }
469
470     public void setSaf_SDBRight(boolean saf_SDBRight) {
471         this.saf_SDBRight = saf_SDBRight;
472     }
473
474     public boolean getSaf_SDBRight() {
475         return saf_SDBRight;
476     }
477
478     public void setDate(LocalDate date) {
479         this.date = date;
480     }
481
482     public LocalDate getDate() {
483         return date;
484     }
485
486     public void setVehicleLinearSpeed(float speed) {
487         this.vehicleLinearSpeed = speed;
488     }
489
490     public float getVehicleLinearSpeed() {
491         return vehicleLinearSpeed;
492     }
493
494     public void setSaf_BSPD(boolean saf_BSPD) {
495         this.saf_BSPD = saf_BSPD;
496     }
497
498     public boolean getSaf_BSPD() {
499         return saf_BSPD;
500     }
501
502     public void setSaf_IMD(boolean saf_IMD) {
503         this.saf_IMD = saf_IMD;
504     }
505
506     public boolean getSaf_IMD() {
507         return saf_IMD;
508     }
509
510     public void setSaf_LNMS(boolean saf_LNMS) {
511         this.saf_LNMS = saf_LNMS;
512     }
513
514     public boolean getSaf_LNMS() {
515         return saf_LNMS;
516     }
517
518     public void setSaf_AMS(boolean saf_AMS) {
519         this.saf_AMS = saf_AMS;
520     }
521
522     public boolean getSaf_AMS() {
523         return saf_AMS;
524     }
525
526     public void setSaf_IS(boolean saf_IS) {
527         this.saf_IS = saf_IS;
528     }
529
530     public boolean getSaf_IS() {
531         return saf_IS;
532     }
533
534     public void setSaf_BOTS(boolean saf_BOTS) {
535         this.saf_BOTS = saf_BOTS;
536     }
537
538     public boolean getSaf_BOTS() {
539         return saf_BOTS;
540     }
541
542     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
543         this.saf_SDB Cockpit = saf_SDB Cockpit;
544     }
545
546     public boolean getSaf_SDB Cockpit() {
547         return saf_SDB Cockpit;
548     }
549
550     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
551         this.saf_SDBLeft = saf_SDBLeft;
552     }
553
554     public boolean getSaf_SDBLeft() {
555         return saf_SDBLeft;
556     }
557
558     public void setSaf_SDBRight(boolean saf_SDBRight) {
559         this.saf_SDBRight = saf_SDBRight;
560     }
561
562     public boolean getSaf_SDBRight() {
563         return saf_SDBRight;
564     }
565
566     public void setDate(LocalDate date) {
567         this.date = date;
568     }
569
570     public LocalDate getDate() {
571         return date;
572     }
573
574     public void setVehicleLinearSpeed(float speed) {
575         this.vehicleLinearSpeed = speed;
576     }
577
578     public float getVehicleLinearSpeed() {
579         return vehicleLinearSpeed;
580     }
581
582     public void setSaf_BSPD(boolean saf_BSPD) {
583         this.saf_BSPD = saf_BSPD;
584     }
585
586     public boolean getSaf_BSPD() {
587         return saf_BSPD;
588     }
589
590     public void setSaf_IMD(boolean saf_IMD) {
591         this.saf_IMD = saf_IMD;
592     }
593
594     public boolean getSaf_IMD() {
595         return saf_IMD;
596     }
597
598     public void setSaf_LNMS(boolean saf_LNMS) {
599         this.saf_LNMS = saf_LNMS;
600     }
601
602     public boolean getSaf_LNMS() {
603         return saf_LNMS;
604     }
605
606     public void setSaf_AMS(boolean saf_AMS) {
607         this.saf_AMS = saf_AMS;
608     }
609
610     public boolean getSaf_AMS() {
611         return saf_AMS;
612     }
613
614     public void setSaf_IS(boolean saf_IS) {
615         this.saf_IS = saf_IS;
616     }
617
618     public boolean getSaf_IS() {
619         return saf_IS;
620     }
621
622     public void setSaf_BOTS(boolean saf_BOTS) {
623         this.saf_BOTS = saf_BOTS;
624     }
625
626     public boolean getSaf_BOTS() {
627         return saf_BOTS;
628     }
629
630     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
631         this.saf_SDB Cockpit = saf_SDB Cockpit;
632     }
633
634     public boolean getSaf_SDB Cockpit() {
635         return saf_SDB Cockpit;
636     }
637
638     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
639         this.saf_SDBLeft = saf_SDBLeft;
640     }
641
642     public boolean getSaf_SDBLeft() {
643         return saf_SDBLeft;
644     }
645
646     public void setSaf_SDBRight(boolean saf_SDBRight) {
647         this.saf_SDBRight = saf_SDBRight;
648     }
649
650     public boolean getSaf_SDBRight() {
651         return saf_SDBRight;
652     }
653
654     public void setDate(LocalDate date) {
655         this.date = date;
656     }
657
658     public LocalDate getDate() {
659         return date;
660     }
661
662     public void setVehicleLinearSpeed(float speed) {
663         this.vehicleLinearSpeed = speed;
664     }
665
666     public float getVehicleLinearSpeed() {
667         return vehicleLinearSpeed;
668     }
669
670     public void setSaf_BSPD(boolean saf_BSPD) {
671         this.saf_BSPD = saf_BSPD;
672     }
673
674     public boolean getSaf_BSPD() {
675         return saf_BSPD;
676     }
677
678     public void setSaf_IMD(boolean saf_IMD) {
679         this.saf_IMD = saf_IMD;
680     }
681
682     public boolean getSaf_IMD() {
683         return saf_IMD;
684     }
685
686     public void setSaf_LNMS(boolean saf_LNMS) {
687         this.saf_LNMS = saf_LNMS;
688     }
689
690     public boolean getSaf_LNMS() {
691         return saf_LNMS;
692     }
693
694     public void setSaf_AMS(boolean saf_AMS) {
695         this.saf_AMS = saf_AMS;
696     }
697
698     public boolean getSaf_AMS() {
699         return saf_AMS;
700     }
701
702     public void setSaf_IS(boolean saf_IS) {
703         this.saf_IS = saf_IS;
704     }
705
706     public boolean getSaf_IS() {
707         return saf_IS;
708     }
709
710     public void setSaf_BOTS(boolean saf_BOTS) {
711         this.saf_BOTS = saf_BOTS;
712     }
713
714     public boolean getSaf_BOTS() {
715         return saf_BOTS;
716     }
717
718     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
719         this.saf_SDB Cockpit = saf_SDB Cockpit;
720     }
721
722     public boolean getSaf_SDB Cockpit() {
723         return saf_SDB Cockpit;
724     }
725
726     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
727         this.saf_SDBLeft = saf_SDBLeft;
728     }
729
730     public boolean getSaf_SDBLeft() {
731         return saf_SDBLeft;
732     }
733
734     public void setSaf_SDBRight(boolean saf_SDBRight) {
735         this.saf_SDBRight = saf_SDBRight;
736     }
737
738     public boolean getSaf_SDBRight() {
739         return saf_SDBRight;
740     }
741
742     public void setDate(LocalDate date) {
743         this.date = date;
744     }
745
746     public LocalDate getDate() {
747         return date;
748     }
749
750     public void setVehicleLinearSpeed(float speed) {
751         this.vehicleLinearSpeed = speed;
752     }
753
754     public float getVehicleLinearSpeed() {
755         return vehicleLinearSpeed;
756     }
757
758     public void setSaf_BSPD(boolean saf_BSPD) {
759         this.saf_BSPD = saf_BSPD;
760     }
761
762     public boolean getSaf_BSPD() {
763         return saf_BSPD;
764     }
765
766     public void setSaf_IMD(boolean saf_IMD) {
767         this.saf_IMD = saf_IMD;
768     }
769
770     public boolean getSaf_IMD() {
771         return saf_IMD;
772     }
773
774     public void setSaf_LNMS(boolean saf_LNMS) {
775         this.saf_LNMS = saf_LNMS;
776     }
777
778     public boolean getSaf_LNMS() {
779         return saf_LNMS;
780     }
781
782     public void setSaf_AMS(boolean saf_AMS) {
783         this.saf_AMS = saf_AMS;
784     }
785
786     public boolean getSaf_AMS() {
787         return saf_AMS;
788     }
789
790     public void setSaf_IS(boolean saf_IS) {
791         this.saf_IS = saf_IS;
792     }
793
794     public boolean getSaf_IS() {
795         return saf_IS;
796     }
797
798     public void setSaf_BOTS(boolean saf_BOTS) {
799         this.saf_BOTS = saf_BOTS;
800     }
801
802     public boolean getSaf_BOTS() {
803         return saf_BOTS;
804     }
805
806     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
807         this.saf_SDB Cockpit = saf_SDB Cockpit;
808     }
809
810     public boolean getSaf_SDB Cockpit() {
811         return saf_SDB Cockpit;
812     }
813
814     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
815         this.saf_SDBLeft = saf_SDBLeft;
816     }
817
818     public boolean getSaf_SDBLeft() {
819         return saf_SDBLeft;
820     }
821
822     public void setSaf_SDBRight(boolean saf_SDBRight) {
823         this.saf_SDBRight = saf_SDBRight;
824     }
825
826     public boolean getSaf_SDBRight() {
827         return saf_SDBRight;
828     }
829
830     public void setDate(LocalDate date) {
831         this.date = date;
832     }
833
834     public LocalDate getDate() {
835         return date;
836     }
837
838     public void setVehicleLinearSpeed(float speed) {
839         this.vehicleLinearSpeed = speed;
840     }
841
842     public float getVehicleLinearSpeed() {
843         return vehicleLinearSpeed;
844     }
845
846     public void setSaf_BSPD(boolean saf_BSPD) {
847         this.saf_BSPD = saf_BSPD;
848     }
849
850     public boolean getSaf_BSPD() {
851         return saf_BSPD;
852     }
853
854     public void setSaf_IMD(boolean saf_IMD) {
855         this.saf_IMD = saf_IMD;
856     }
857
858     public boolean getSaf_IMD() {
859         return saf_IMD;
860     }
861
862     public void setSaf_LNMS(boolean saf_LNMS) {
863         this.saf_LNMS = saf_LNMS;
864     }
865
866     public boolean getSaf_LNMS() {
867         return saf_LNMS;
868     }
869
870     public void setSaf_AMS(boolean saf_AMS) {
871         this.saf_AMS = saf_AMS;
872     }
873
874     public boolean getSaf_AMS() {
875         return saf_AMS;
876     }
877
878     public void setSaf_IS(boolean saf_IS) {
879         this.saf_IS = saf_IS;
880     }
881
882     public boolean getSaf_IS() {
883         return saf_IS;
884     }
885
886     public void setSaf_BOTS(boolean saf_BOTS) {
887         this.saf_BOTS = saf_BOTS;
888     }
889
890     public boolean getSaf_BOTS() {
891         return saf_BOTS;
892     }
893
894     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
895         this.saf_SDB Cockpit = saf_SDB Cockpit;
896     }
897
898     public boolean getSaf_SDB Cockpit() {
899         return saf_SDB Cockpit;
900     }
901
902     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
903         this.saf_SDBLeft = saf_SDBLeft;
904     }
905
906     public boolean getSaf_SDBLeft() {
907         return saf_SDBLeft;
908     }
909
910     public void setSaf_SDBRight(boolean saf_SDBRight) {
911         this.saf_SDBRight = saf_SDBRight;
912     }
913
914     public boolean getSaf_SDBRight() {
915         return saf_SDBRight;
916     }
917
918     public void setDate(LocalDate date) {
919         this.date = date;
920     }
921
922     public LocalDate getDate() {
923         return date;
924     }
925
926     public void setVehicleLinearSpeed(float speed) {
927         this.vehicleLinearSpeed = speed;
928     }
929
930     public float getVehicleLinearSpeed() {
931         return vehicleLinearSpeed;
932     }
933
934     public void setSaf_BSPD(boolean saf_BSPD) {
935         this.saf_BSPD = saf_BSPD;
936     }
937
938     public boolean getSaf_BSPD() {
939         return saf_BSPD;
940     }
941
942     public void setSaf_IMD(boolean saf_IMD) {
943         this.saf_IMD = saf_IMD;
944     }
945
946     public boolean getSaf_IMD() {
947         return saf_IMD;
948     }
949
950     public void setSaf_LNMS(boolean saf_LNMS) {
951         this.saf_LNMS = saf_LNMS;
952     }
953
954     public boolean getSaf_LNMS() {
955         return saf_LNMS;
956     }
957
958     public void setSaf_AMS(boolean saf_AMS) {
959         this.saf_AMS = saf_AMS;
960     }
961
962     public boolean getSaf_AMS() {
963         return saf_AMS;
964     }
965
966     public void setSaf_IS(boolean saf_IS) {
967         this.saf_IS = saf_IS;
968     }
969
970     public boolean getSaf_IS() {
971         return saf_IS;
972     }
973
974     public void setSaf_BOTS(boolean saf_BOTS) {
975         this.saf_BOTS = saf_BOTS;
976     }
977
978     public boolean getSaf_BOTS() {
979         return saf_BOTS;
980     }
981
982     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
983         this.saf_SDB Cockpit = saf_SDB Cockpit;
984     }
985
986     public boolean getSaf_SDB Cockpit() {
987         return saf_SDB Cockpit;
988     }
989
990     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
991         this.saf_SDBLeft = saf_SDBLeft;
992     }
993
994     public boolean getSaf_SDBLeft() {
995         return saf_SDBLeft;
996     }
997
998     public void setSaf_SDBRight(boolean saf_SDBRight) {
999         this.saf_SDBRight = saf_SDBRight;
1000     }
1001
1002     public boolean getSaf_SDBRight() {
1003         return saf_SDBRight;
1004     }
1005
1006     public void setDate(LocalDate date) {
1007         this.date = date;
1008     }
1009
1010     public LocalDate getDate() {
1011         return date;
1012     }
1013
1014     public void setVehicleLinearSpeed(float speed) {
1015         this.vehicleLinearSpeed = speed;
1016     }
1017
1018     public float getVehicleLinearSpeed() {
1019         return vehicleLinearSpeed;
1020     }
1021
1022     public void setSaf_BSPD(boolean saf_BSPD) {
1023         this.saf_BSPD = saf_BSPD;
1024     }
1025
1026     public boolean getSaf_BSPD() {
1027         return saf_BSPD;
1028     }
1029
1030     public void setSaf_IMD(boolean saf_IMD) {
1031         this.saf_IMD = saf_IMD;
1032     }
1033
1034     public boolean getSaf_IMD() {
1035         return saf_IMD;
1036     }
1037
1038     public void setSaf_LNMS(boolean saf_LNMS) {
1039         this.saf_LNMS = saf_LNMS;
1040     }
1041
1042     public boolean getSaf_LNMS() {
1043         return saf_LNMS;
1044     }
1045
1046     public void setSaf_AMS(boolean saf_AMS) {
1047         this.saf_AMS = saf_AMS;
1048     }
1049
1050     public boolean getSaf_AMS() {
1051         return saf_AMS;
1052     }
1053
1054     public void setSaf_IS(boolean saf_IS) {
1055         this.saf_IS = saf_IS;
1056     }
1057
1058     public boolean getSaf_IS() {
1059         return saf_IS;
1060     }
1061
1062     public void setSaf_BOTS(boolean saf_BOTS) {
1063         this.saf_BOTS = saf_BOTS;
1064     }
1065
1066     public boolean getSaf_BOTS() {
1067         return saf_BOTS;
1068     }
1069
1070     public void setSaf_SDB Cockpit(boolean saf_SDB Cockpit) {
1071         this.saf_SDB Cockpit = saf_SDB Cockpit;
1072     }
1073
1074     public boolean getSaf_SDB Cockpit() {
1075         return saf_SDB Cockpit;
1076     }
1077
1078     public void setSaf_SDBLeft(boolean saf_SDBLeft) {
1079         this.saf_SDBLeft = saf_SDBLeft;
1080     }
1081
1082     public boolean getSaf_SDBLeft() {
1083         return saf_SDBLeft;
1084     }
1085
1086     public void setSaf_SDBRight(boolean saf_SDBRight) {
1087         this.saf_SDBRight = saf_SDBRight;
1088     }
1089
1090     public boolean getSaf_SDBRight() {
1091         return saf_SDBRight;
1092     }
1093
1094     public void setDate(LocalDate date) {
1095         this.date = date;
1096     }
1097
1098     public LocalDate getDate() {
1099         return date;
1100     }
1101
1102     public void setVehicleLinearSpeed(float speed) {
1103         this.vehicleLinearSpeed = speed;
1104     }
1105
1106     public float getVehicleLinearSpeed() {
1107         return vehicleLinearSpeed;
1108     }
1109
1110     public void setSaf_BSPD(boolean saf_BSPD) {
1111         this.saf_BSPD = saf_BSPD;
1112     }
1113
1114     public boolean getSaf_BSPD() {
1115         return saf_BSPD;
1116     }
1117
1118     public void setSaf_IMD(boolean saf_IMD) {
1119         this.saf_IMD = saf_IMD;
1120     }
1121
1122     public boolean getSaf_IMD() {
1123         return saf_IMD;
1124     }
1125
1126     public void setSaf_LNMS(boolean saf_LNMS) {
1127         this.saf_LNMS = saf_LNMS;
1128     }
1129
1130     public boolean getSaf_LNMS() {
1131         return saf_LNMS;
1132     }
1133
1134     public void setSaf_AMS(boolean saf_AMS) {
1135         this.saf_AMS = saf_AMS;
1136     }
1137
1138     public boolean getSaf_AMS() {
1139         return saf_AMS;
1140     }
1141
1142     public void setSaf_IS(boolean saf_IS) {
1143         this.saf_IS = saf_IS;
1144     }
1145
1146     public
```

10.0.2 TestFx

Abbiamo anche provato a testare la classe Tab1, della view, per testare la selezione delle date che ci permettono di printare a schermo la tabella contenente lo storico dei dati.

A causa di incompatibilità con la versione di javafx utilizzata i test non possono essere fatti usando TestFx.

Nel caso in cui fosse risultato fattibile, avremmo utilizzato i seguenti casi di test, visibili nel codice all'interno della classe Tab1Test:



```
65 @Test
66 public void test() {
67     Platform.runLater(new Runnable() {
68         public void run() {
69             Tab1 tab01 = new Tab1();
70             /*test 01,
71              * selezione data iniziale < data finale e mi aspetto che tab01.getStartDate(prima, dopo) prenda una data non nulla,
72              * in realtà ho sbagliato a scrivere la data iniziale, anziché LocalDate.of(2024, 3, 11) ho scritto LocalDate.of(2024, 3, 13)
73              * quindi durante il test vedrei null */
74             /*
75             DatePicker prima01 = new DatePicker(LocalDate.of(2024, 3, 13)); //anno, mese, giorno
76             DatePicker dopo01 = new DatePicker(LocalDate.of(2024, 3, 12));
77             tab01.setInizio(prima01);
78             tab01.setFine(dopo01);
79             assertNotNull(tab01.getStartDate(prima, dopo));
80
81
82             //test02, selezione data iniziale > data finale, mi aspetto null come risultato del tab01.getStartDate(prima, dopo)
83             DatePicker prima02 = new DatePicker(LocalDate.of(2024, 3, 13)); //anno, mese, giorno
84             DatePicker dopo02 = new DatePicker(LocalDate.of(2024, 3, 12));
85             tab01.setInizio(prima02);
86             tab01.setFine(dopo02);
87             assertNull(tab01.getStartDate(prima, dopo));
88
89             //test03, selezione data iniziale = data finale, mi aspetto notNull come risultato del tab01.getStartDate(prima, dopo)
90             DatePicker prima03 = new DatePicker(LocalDate.of(2024, 3, 12)); //anno, mese, giorno
91             DatePicker dopo03 = new DatePicker(LocalDate.of(2024, 3, 12));
92             tab01.setInizio(prima03);
93             tab01.setFine(dopo03);
94             assertNull(tab01.getStartDate(prima, dopo));
95
96             //test04, selezione data iniziale < data finale, mi aspetto notNull come risultato del tab01.getStartDate(prima, dopo)
97             DatePicker prima04 = new DatePicker(LocalDate.of(2024, 3, 11)); //anno, mese, giorno
98             DatePicker dopo04 = new DatePicker(LocalDate.of(2024, 3, 12));
99             tab01.setInizio(prima04);
100             tab01.setFine(dopo04);
101             assertNotNull(tab01.getStartDate(prima, dopo));
102         }
103     });
104 }
```

Diagrammi UML

Diagramma attività

Con questo diagramma vogliamo rappresentare le attività che l'utente e il software devono svolgere per ottenere l'esportazione del file csv contenente i dati presenti in tabella.

Questa attività inizia con il click del pulsante denominato "csv export".

Questo aprirà una finestra nella quale possiamo fare diverse attività:

- inserire il nome del file, attività obbligatoria per il salvataggio
- selezione del percorso per il salvataggio del file, attività opzionale, nel caso in cui non si selezioni manualmente un percorso, il file verrà salvato nella posizione predefinita
- pressione pulsante salva, esporta il file solo se precedentemente è stato inserito un nome per il file.

Non c'è un controllo sui dati, se non sono presenti dati in tabella il file esportato conterrà solo l'elenco di sensori, senza alcun dato associato.

Il diagramma termina con l'esportazione del file.

- annullare l'operazione, tramite il tasto "annulla" o il tasto "X", chiude la finestra senza produrre output, quindi termina il diagramma

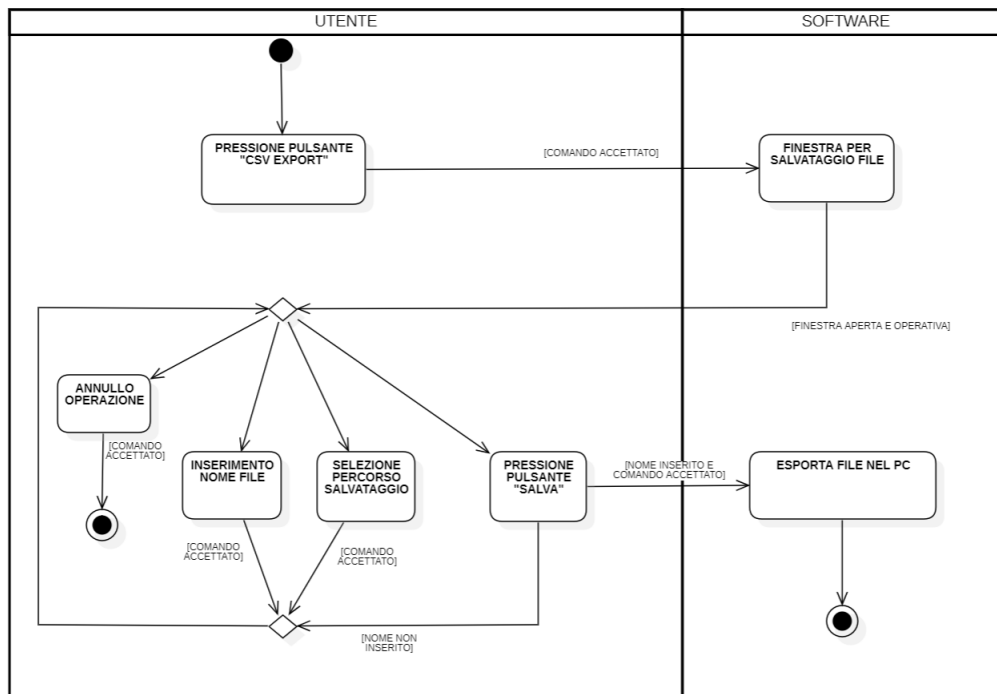


Diagramma casi d'uso

Con il diagramma dei casi d'uso vogliamo descrivere il sistema, ponendo l'attenzione su chi interagisce con esso e cosa può fare ognuno di essi.

Abbiamo tre attori principali che interagiscono con il sistema, sono:

- Microcontrollori: devono poter inviare e ricevere i dati poichè abbiamo un trasmettitore e un ricevitore
- Utente: deve poter richiedere la comunicazione con i microcontrollori, deve poter visualizzare i dati , in real time e nello storico, e deve poter esportare i dati.
- Database: deve gestire i dati nelle operazioni di salvataggio in locale e di esportazione

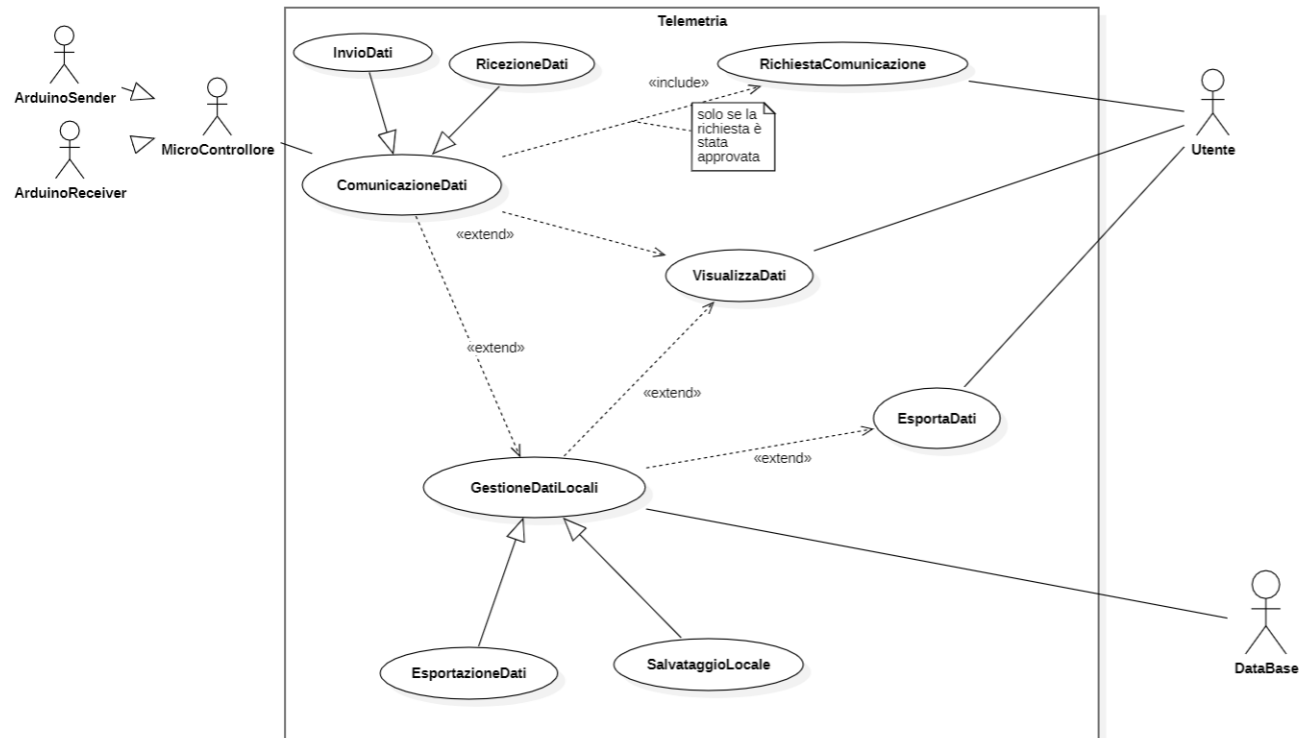


Diagramma di sequenza

Con il seguente diagramma vogliamo rappresentare il processo di scambio dei dati tra il programma e i microcontrollori.

Questo processo si divide in diverse fasi:

- fase 1: dobbiamo verificare che ci siano microcontrollori attivi sulle porte COM, se non ci sono, il processo si interrompe e l'utente dovrà ricominciare dall'inizio
- fase 2: L'utente dovrà selezionare la porta COM desiderata per poter avviare la connessione
- fase 3, Loop di 200ms:
ogni 200ms vengono ricevuti i dati.
si controlla che la connessione con la porta sia ancora attiva (potrebbe esserci un problema con il microcontrollore), se non attiva, termina il processo.
si controlla che l'utente non abbia richiesto la disconnessione della comunicazione con la porta COM, in tal caso termina il processo.

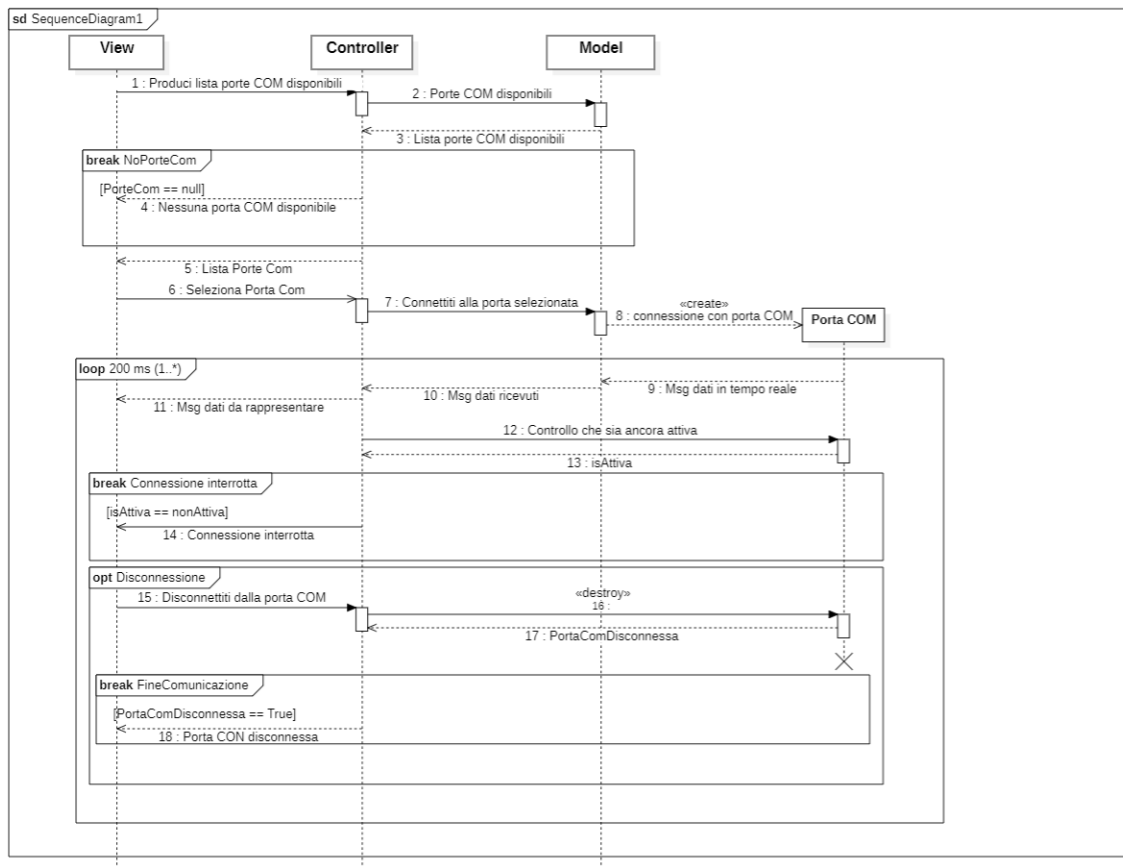


Diagramma della macchina a stati

Con il seguente diagramma vogliamo rappresentare gli stati del sistema durante l'operazione di caricamento dei dati in tabella.

- il primo stato è quello nel quale troviamo l'applicazione aperta nella home page, con la tabella di dati vuota e le date impostate sui valori predefiniti (selezionate automaticamente guardando: **Data inizio** cioè la data attuale del computer e **Data fine** cioè il giorno precedente)

Tramite le operazioni che farà l'utente potremmo trovarci in diversi stati:

- se sceglie la data iniziale tramite l'apposita icona, saremo nello stato nel quale si apre il calendario che permette di selezionare la data iniziale dalla quale prendere i dati.
- se sceglie la data finale tramite l'apposita icona, saremo nello stato nel quale si apre il calendario che permette di selezionare la data finale fino alla quale prendere i dati.
- se seleziona l'icona "load" saremo nello stato nel quale il sistema tenterà di caricare i dati in tabella, quest'ultimo stato porterà il sistema in ulteriori possibili stati:
 - stato nel quale appare un popup a schermo, se le date selezionate non rispettano i vincoli imposti (cioè la data finale deve essere successiva a quella iniziale)
 - primo stato , con date selezionate (se rispettano i vincoli) e tabella vuota (in quelle date non c'erano dati salvati)
 - stato nel quale nella tabella sono stati caricati i dati
- il sistema termina se l'utente seleziona l'icona per la chiusura dell'applicazione

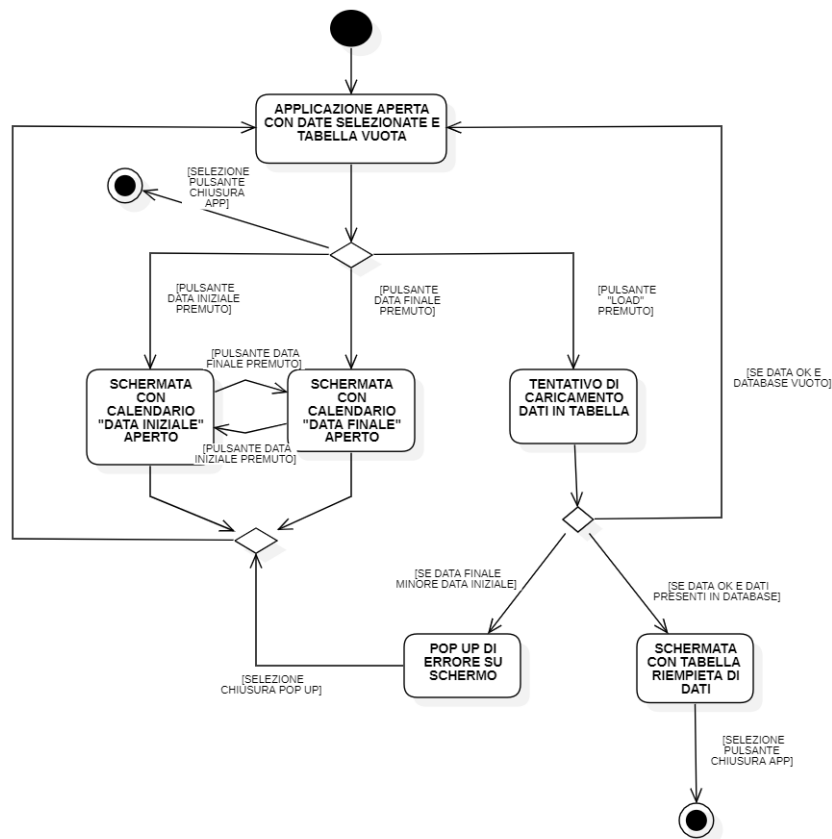


Diagramma delle classi

Nel diagramma delle classi abbiamo rappresentato un pattern MVC (Model-View-Controller) che rappresenta l'architettura di partenza, che utilizziamo per la programmazione del nostro software.

- **view:** nel package sono presenti diverse classi, View è la classe che si occupa dell'interfaccia grafica completa e dell'interazione dell'utente con i componenti grafici del software (GUI). L'interfaccia è definita grazie all'uso di contenitori (classi di javafx chiamate Pane) che ci permettono di comporre il layout tramite contenitori standardizzati. Noi usiamo `BorderPane` per dividere l'interfaccia in 3 sezioni, `TopPane`, `CenterPane` e `BottomPane`, all'interno dei quali inseriamo, sempre tramite `Pane`, elementi grafici disposti secondo le specifiche definite con i clienti.
- **model:** package contenente le classi che gestiscono tutte le funzionalità logiche del programma. All'interno del Model è riconoscibile una semplice struttura a strati per cui il model si appoggia sui servizi sottostanti forniti dal gestore del database e dal gestore della porta seriale.
- **controller:** package che contiene le classi che hanno il compito da fare da intermediario tra Model e View: si occupa di richiamare le funzioni del Model corrispondenti ai comandi del View e gestisce l'aggiornamento della GUI con i dati provenienti dal Model.
- **telemetry:** è il package che contiene la classe principale (`Main`) che si occupa dell'inizializzazione del software e dell'avvio.

